lavmain.cpp:

Az std::list-hez biztosít két extensiont/nézetet, az első az array_view, ami egy std::list-et tömbként láttat, nem változik az elemszám, 0-tól indexelhetőek az elemei konstans időben. A listát megkapja létrehozáskor, írni/olvasni lehet az at tagfüggvény, illetve indexelő operátor segítségével. A konstans időt úgy kell elérni, hogy a második template paraméter meghatározza azt a hatékonyan indexelhető konténert, amiben a lista elemeit elérő iterátorokat tárolni kell. A második nézet a vector_view, ami az array_view altípusa, de támogatja a push_back-elést is, ilyenkor maga az eredeti listához is push_back-elni kell az elemet. 5-ösért ne kelljen megadni az iterátorok konténerét, valamint a sort tagfüggvényeken keresztül nézetileg rendezhetőek az elemek, azaz a nézet indexelésén keresztül rendezetten látszódnak az adatok, de a listában maradnak az eredeti helyükön.

solmain.cpp:

A self_organizing_list osztály template az std::list-en történő keresések optimalizálása végett bevezeti a contains metódust, ami egy bool-lal tér vissza és egy template paraméter típusú elemet keres a konstruktorban megkapott listán. Különféle heurisztikák alapján átrendezi a list elemeit, hogy a további keresések minél hatékonyabbak lehessenek. Két heurisztika, amit meg kell megvalósítani: move_to_front, ami az utoljára sikeresen keresett elemet a list elejére teszi adatvesztés nélkül, illetve a frequency_list, ami sikeres keresésekről statisztikát vezet és minél több sikeres keresést végzett egy adott elemről, annál előrébb kerül a list-ben. 4-esért és 5-ösért a heurisztikákban legyenek predikátum szerinti keresések (contains_if) is, ezeket elég csak a heurisztikákban megvalósítani.

cbmain.cpp:

A circular_buffer osztály template egy allokáció-mentes buffert valósít meg termelő-fogyasztó problémához: létrehozáskor megkap egy tömböt, ami a kezdetben üres buffert jelenti. Az insert-tel lehet berakni egy elemet, read-del lehet kiolvasni. Arra kell figyelni, hogy ne írjunk felül még feldolgozatlan elemet, csak akkor tároljunk, ha van még szabad kapacitás. Ha tömb megtelt és van még szabad kapacitás, akkor az elejére újra el lehet kezdeni írni elemeket. A capacity megadja a kapacitást, a size pedig a bufferben lévő még feldolgozatlan elemek számát. 5-ösért egy bool specializációt kell biztosítani, ami egy int tömbben helyiértékenként tárolja a logikai értékeket.

sstvmain.cpp:

A set_stack_view egy olyan osztály template, ami hagyományos std::set-eket láttat stack-ként, vagy másképpen egy olyan stack-et kell megvalósítani, ami egy std::set-ben tárolja az adatokat. Létrehozáskor megkapja azt a std::set objektumot, amit stack-ként fog kezelni, a set az addigi állapotától függetlenül üressé válik, felkészül a stack-ként való felhasználásra, továbbá a nézet lekerülésekor visszaáll erre az állapotára. A stack-be push-sal tudunk adatokat tárolni, az utoljára _sikeresen_ eltárolt adat lesz a verem tetején. (Nyilván a set-ben egy elem csak egyszer szerepelhet, így nem minden push tárol el érdemben elemet. ). A top az utolsó sikeresen eltárolt

elemet adja vissza, a pop törli. Az operator+=-vel további set elemeit lehet sorban megpróbálni a stack-hez push-olni, 4-esért akár olyan rendezéssel, ami nem egyezik meg a kiindulási set rendezésével. 4-esért működjön együtt tetszőleges rendezést felhasználó std::set-tel, 5-ösért pedig tetszőleges intervallumban lévő elemeket lehessen push-olni, továbbá működjön az std::stack konverzió is, a stack mögötti konténer paraméterezhetőségével együtt.

assocmain.cpp:

Az association osztály template feladata, hogy már létező szabványkönyvtári konténerekhez biztosítson egy utólagos tag-elési, asszociációs réteget. Ennek template paraméterei: a kulcs típusa, a szabványkönyvtári konténer típusa, az ebben tárolt típus, illetve a kulcsok egyezéséhez használt opcionális funktor típusa. Létrehozáskor megkapja a konténert, az associate-tel lehet a konténer egy eleméhez egy kulcsot rendelni és a find-dal lehet lekérdezni a tag-hez rendelt értéket, has-zel lekérdezhető, hogy egy tag létezik-e, illetve az operator[]-a map-jellegű, felvehető azon keresztül a konténerbe, ha olyan kulcsra hivatkozik, ami még nem volt a konténerhez rendelve. Ilyenkor át is lehet írni a konténer elemét. 4-esért ne kelljen megadni a konténer tárolt elem típusát, de akár lehessen felhasználói tag egyezést megadni. 5-ösért működjön együtt tetszőleges rendezésű set-tel olyan módon, hogy az operator[] ne írhasson át elemet utólagosan. A template specializációban lévő kódújrafelhasználás az associate_common ősosztályon keresztül történjen. Iterátor invalidációra nem kell felkészülni.

ddbmain.cpp:

A double_circular_buffer egy olyan sablon osztály, amely a termelő-fogyasztó patternhez biztosít segítséget olyan módon, hogy a konstruktorban átadott buffert két folyamat (A-val, illetve B-vel jelölve) számára biztosít allokáció-mentes buffert. Kezdetben a buffert a két folyamat egyenlő részre osztja ketté, az első felét az A folyamat használja, a második felét a B folyamat használja. Kezdetben insert_A és az insert_B tagfüggvények úgy működnek, hogy nem írnak felül meglévő adatot, ha nincs szabad kapacitás, akkor eldobják a tárolandó elemet. Amikor viszont a mindkét read... művelet elkezdte feldolgozni a bufferben lévő adatokat, akkor a teljes kapacitás nem feltétlenül marad egyenlő arányban kettéosztva, hanem a sikeres read műveletek arányában osztja ketté. Ilyenkor nyilván veszhetnek adatok, de nincsenek nagyon speciális corner-case-ek a tesztesetek között, nem kell algoritmikusan túlgondolni a dolgot. 4-esért a buffer méretét ne kelljen kézzel átadni, működjön együtt egy vector-ral is. 5-ösért az insert-eknek legyen intervallumot váró változata is.